# Neural Tangent Kernels for Axis-Aligned Tree Ensembles

Ryuichi Kanoh[1,2], Mahito Sugiyama[1,2]
[1]National Institute of Informatics, [2]The Graduate University for Advanced Studies, SOKENDAI

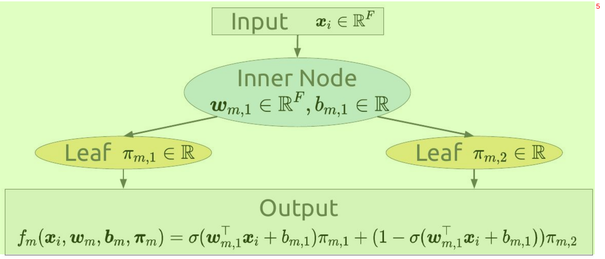
ICML International Conference On Machine Learning

## Contribution [1]

- We formulate the Neural Tangent Kernel (NTK) induced by infinitely many axis-aligned differentiable tree ensembles. [2]

## Differentiable Decision Tree [3]

- By employing a sigmoid-like function, we can relax the splitting into a continuous form, which allows the use of gradient descent. [4]


 [5]

## Neural Tangent Kernel for Tree Ensembles [6]

- The NTK emerges in the context of functional gradient descent. [7]

### Functional Gradient Descent [8]

$$\frac{\partial f(\boldsymbol{x}, \boldsymbol{\theta})}{\partial t} = -\eta \underbrace{\frac{\partial f(\boldsymbol{x}, \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \frac{\partial f(\boldsymbol{x}, \boldsymbol{\theta})}{\partial \boldsymbol{\theta}}^\top}_{\text{Neural Tangent Kernel}} (f(\boldsymbol{x}, \boldsymbol{\theta}) - \boldsymbol{y})$$ [9]

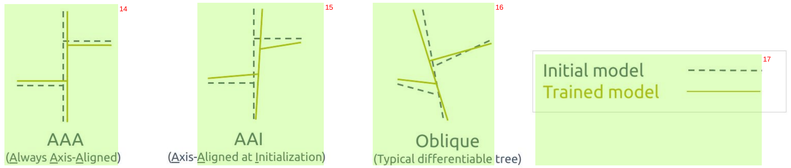
- It does not change during training when considering infinitely wide neural networks. [10]
- Then, functional behavior becomes analytically tractable.

- By handling the number of trees like the width of neural networks, we can apply NTK theory to differentiable tree ensembles. [11]
  - Proved in our previous work: Kanoh & Sugiyama (ICLR 2022)

- In this work, we extend the NTK concept for axis-aligned trees.

## Axis-Aligned Constraint [12]

- We consider two axis-aligned cases: AAA and AAI. [13]
  - If $\boldsymbol{w}_{m,n}$ is in one-hot representation, splitting is axis-aligned.


 [14] [15] [16] [17]

## Closed-form NTK formula for Axis-Aligned Trees [18]

### NTK for Infinite Ensembles of Axis-Aligned Trees [19]

$$\Theta^{\text{AxisAligned}}(\boldsymbol{x}_i, \boldsymbol{x}_j) := \lim_{M \to \infty} \widehat{\Theta}_0^{\text{AxisAligned}}(\boldsymbol{x}_i, \boldsymbol{x}_j)$$
$$= \underbrace{\sum_{\ell=1}^{\mathcal{L}} \sum_{s \in h(a_\ell)} \left( \Sigma_{\{i,j\},s} \dot{\mathcal{T}}_{\{i,j\},s} \prod_{t \in h(a_\ell) \setminus \{s\}} \mathcal{T}_{\{i,j\},t} \right)}_{\text{contribution from internal nodes}} + \underbrace{\sum_{\ell=1}^{\mathcal{L}} \prod_{u \in h(a_\ell)} \mathcal{T}_{\{i,j\},u}}_{\text{contribution from leaves}}$$ [20]

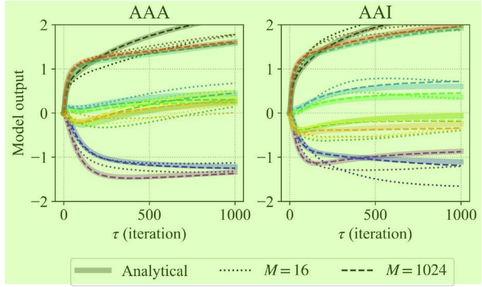
$$\Sigma_{\{i,j\},s} = \begin{cases} x_{i,s} x_{j,s} + \beta^2 & \text{if AAA is used} \\ \boldsymbol{x}_i^\top \boldsymbol{x}_j + \beta^2 & \text{if AAI is used} \end{cases}$$

$$\mathcal{T}_{\{i,j\},s} = \frac{1}{2\pi} \arcsin\left( \frac{\alpha^2 (x_{i,s} x_{j,s} + \beta^2)}{\sqrt{(\alpha^2 (x_{i,s}^2 + \beta^2) + 0.5)(\alpha^2 (x_{j,s}^2 + \beta^2) + 0.5)}} \right) + \frac{1}{4}$$

$h(a_\ell) \subset \mathbb{N}$ : The set of feature indices used in splits of the root-to-leaf path $a_\ell$.

$$\dot{\mathcal{T}}_{\{i,j\},s} = \frac{\alpha^2}{\pi} \frac{1}{\sqrt{\left(1 + 2\alpha^2 (x_{i,s}^2 + \beta^2)\right)(1 + 2\alpha^2 (x_{j,s}^2 + \beta^2)) - 4\alpha^4 (x_{i,s} x_{j,s} + \beta^2)^2}}$$

- Training behavior is now analytically tractable using our formula. [21]


 [22]

Output dynamics for test data points. Each line color corresponds to each data point. [23]

## Ensembles of Various Trees [24]

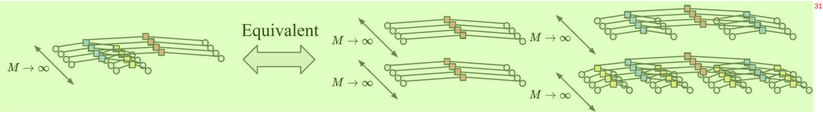
- The NTK induced by an ensemble of various tree architectures is equal to the sum of the NTKs induced by each architecture. [25]
  - ρ is the proportion of the corresponding tree architecture within the ensemble. [26]

### NTK Property for Mixed Trees [27]

$$\Theta^{\text{Mixed}}(\boldsymbol{x}_i, \boldsymbol{x}_j) = \rho_1 \Theta^{\text{Architecture1}}(\boldsymbol{x}_i, \boldsymbol{x}_j) + \rho_2 \Theta^{\text{Architecture2}}(\boldsymbol{x}_i, \boldsymbol{x}_j) + \dots$$ [28]

## Insight on Oblivious Trees [29]

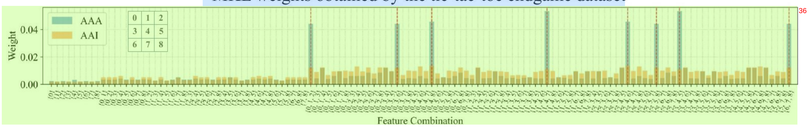
- Any non-oblivious trees can be transformed into oblivious trees that induce exactly the same limiting NTK. [30]
  - There is no need to consider combinations of complex trees, and it is sufficient to consider only combinations of oblivious trees.


 [31]

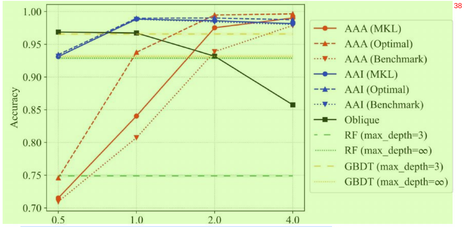
## Multiple Kernel Learning (MKL) as Architecture Search [32]

- MKL enables tree architecture search based on convex optimization. [33]
  - Suitable features for AAA and AAI can be different. [34]

MKL weights obtained by the tic-tac-toe endgame dataset [35]

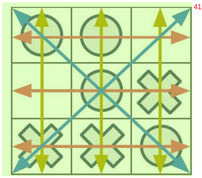

 [36]

Classification accuracy on tic-tac-toe endgame dataset [37]


 [38]

Hardness of the sigmoid used in a splitting [39]

Interactions for tic-tac-toe endgame [40]


[41]